

Guide de commandes (GoogleMLKit)

Date: 2026-05-20

Contexte: Windows + PowerShell, venv Python 3.12 dans `backend/.venv`.

1) Se placer au bon endroit

Toujours partir de la racine du projet:

```
Code (powershell)
cd C:\Users\DELL\Downloads\GoogleMLKit
```

2) Activer le venv Python (backend)

Si PowerShell bloque l'activation (scripts), autoriser seulement pour le process courant:

```
Code (powershell)
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

Activer le venv:

```
Code (powershell)
& ".\backend\.venv\Scripts\Activate.ps1"
```

Vérifier la version de Python (doit être 3.12.x):

```
Code (powershell)
python --version
```

3) Installer les dépendances minimales pour les tests

Depuis la racine du repo:

```
Code (powershell)
python -m pip install -r .\backend\requirements-ci.txt
```

(Optionnel) Mettre à jour pip:

```
Code (powershell)
python -m pip install -U pip
```

4) Lancer les tests (backend)

Important: se placer dans le dossier `backend/` pour exécuter les tests.

```
Code (powershell)
cd .\backend
```

Tests rapides:

```
Code (powershell)
python -m pytest -q tests
```

5) Lancer les tests + couverture (commande "qui passe")

La couverture doit être calculée uniquement sur `api\_inference.py`.

```
Code (powershell)
python -m pytest -q tests --cov=api_inference --cov-report=term-missing --cov-fail-under=70
```

Pourquoi pas `--cov=.` ?

- `--cov=.` compte *tout* le code Django dans `apps/`, `config/`, etc.
- Comme ces modules ne sont pas testés ici, la couverture totale tombe vers ~5%.

6) Comprendre les warnings scikit-learn (InconsistentVersionWarning)

Tu as:

- pickle entraîné/sauvegardé avec scikit-learn `1.7.2`
- environnement actuel avec scikit-learn `1.8.0`

Ce warning ne fait pas échouer les tests, mais il signifie:

- le modèle désérialisé peut être incompatible (rare, mais possible)
- pour le supprimer: ré-entraîner et re-sauvegarder le modèle avec la *même* version de scikit-learn que ton environnement.

(Optionnel) Pour voir la version installée:

Code (powershell)

```
python -c "import sklearn; print(sklearn.__version__)"
```

7) Générer les présentations (PPTX)

Revenir à la racine du projet:

Code (powershell)

```
cd ..
```

Installer les dépendances PPT (si nécessaire):

Code (powershell)

```
python -m pip install -U python-pptx Pillow
```

Générer les fichiers:

Code (powershell)

```
python .\make_ppt.py
```

Lister les PPTX générés:

Code (powershell)

```
Get-ChildItem -Path . -Filter "*.pptx" | Select-Object Name,Length
```

Résultat attendu:

- `EcoSmartClassifier\_presentation\_7min.pptx`
- `EcoSmartClassifier\_presentation\_12min.pptx`

8) Astuces (commandes utiles)

Lister les packages installés dans le venv:

Code (powershell)

```
python -m pip freeze | Out-File .\pip_freeze.txt
```

Lancer un seul test (exemple):

Code (powershell)

```
python -m pytest -q tests\test_api_inference.py -k "test_"
```

9) Modèle de section “mes autres commandes”

Tu peux compléter ici:

- Objectif: ...
- Dossier: racine / backend / frontend
- Commande:

Code (powershell)

```
# ...
```